REAL: Efficient Distributed Training of Dynamic GNNs with Reuse-Aware Load Balancing Scheduling

Zihao Fan[†], Yunzhuo Liu[†], Tian Guo[‡], Bo Jiang[†*]
[†]*Shanghai Jiao Tong University* [‡]*Worcester Polytechnic Institute*

***Abstract*—Dynamic Graph Neural Networks (DGNNs) enable learning from evolving graphs by capturing both structural and temporal dependencies. However, training DGNNs at scale is challenging due to three conflicting requirements: maximizing resource utilization, minimizing data transfers, and maximizing data reuse. Existing systems typically optimize one objective at the expense of the others, leading to communication bottlenecks, load imbalance, or redundant computation. We present REAL, a reuse-aware distributed system for DGNN training that jointly addresses these challenges. REAL minimizes data transfers via snapshot-group scheduling, and leverages an incremental aggregation mechanism to decouple computation from structural updates, effectively pruning redundant I/O and FLOPs via snapshot- and group-level reuse. We further design two complementary schedulers: REAL-L, an ILP-based optimizer for globally optimal planning, and REAL-G, a scalable greedy heuristic. Experiments on six dynamic graph datasets and four DGNN models show that REAL-L and REAL-G achieve $1.11\times$–$3.26\times$ and $1.10\times$–$3.16\times$ speedup over the strongest baseline.**

I. INTRODUCTION

Dynamic graphs are graphs whose structures and attributes change continuously over time. Dynamic Graph Neural Networks (DGNNs) have emerged as state-of-the-art methods for processing dynamic graphs, exhibiting a strong ability to capture both structural and temporal dependencies [1–18]. Depending on the event model, dynamic graphs are categorized into discrete-time dynamic graphs (DTDGs) and continuous-time dynamic graphs (CTDGs). Discrete-time dynamic graphs evolve via snapshot-based updates, whereas continuous-time dynamic graphs are driven by continuous event streams. DGNNs are categorized similarly according to the dynamic graphs they process. In this work, we focus on DGNNs for DTDGs, which model temporal dynamics via discrete snapshots and are widely adopted in both real-world applications [8, 19, 20] and mainstream graph learning frameworks [21–23].

While significant efforts have been made to improve DGNN training efficiency [24–30], the escalating scale of graph datasets renders distributed training across multiple GPUs increasingly important [31–34]. Distributed DGNN training requires simultaneously (1) *maximizing resource utilization*, (2) *minimizing data transfer overhead* (both cross-GPU and CPU-GPU), and (3) *maximizing data reuse*, which constitute three often conflicting objectives. Existing distributed systems navigate these trade-offs with limited success. ESDG [31] distributes adjacent snapshots across GPUs; this necessitates expensive hidden state transfers and, due to RNN sequential dependencies, forces GPUs to idle while awaiting the previous hidden states, degrading utilization. BLAD [32] circumvents such transfer overhead by processing each snapshot group on the same GPU while assigning different groups on different GPUs. However, variations across groups lead to load imbalance, which also results in inefficient resource utilization. Similarly, DGC [33] employs chunk-based partitioning to jointly optimize load balance and communication; however, prioritizing partition quality frequently exacerbates load imbalance due to the intrinsic conflict between these objectives. DynaHB [34] leverages CPU vertex caching to balance workloads and eliminate inter-GPU communication, yet this incurs prohibitive CPU-GPU data transfer overhead, shifting the bottleneck to the PCIe bus. Moreover, these distributed frameworks overlook the opportunities for data reuse. While prior studies [25, 27, 35, 36] demonstrate that leveraging topological similarity between adjacent snapshots significantly reduces redundant computation and I/O, exploiting this reuse introduces computational irregularity due to varying redundancy rates across snapshots. This irregularity exacerbates load imbalance in distributed settings, yet no existing system co-optimizes such data reuse with workload distribution.

To solve this challenging problem, we design REAL, a reuse-aware distributed DGNN training system that jointly optimizes the three objectives above and balances the trade-offs among them. Its benefits are most pronounced on DTDG workloads with temporal smoothness, *i.e.,* substantial structural overlap between adjacent snapshots, which is common in real-world evolving graphs. Our key insight is that *different snapshot groups are treated as independent samples in DGNN training*, which allows us to flexibly combine groups and schedule them in an arbitrary order. **First**, REAL adopts *snapshot groups* as the basic scheduling unit, which naturally avoids cross-GPU data transfers. **Second**, REAL enables topology-aware data reuse at both snapshot and group levels. REAL designs new operators that explicitly exploit structural similarities between snapshots and overlapping snapshots across groups, significantly reducing redundant computation and data loading overhead. In addition, REAL overlaps data transfer with computation across heterogeneous resources in a pipelined manner, further improving resource utilization.

*Corresponding author.

SC26, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-4789-7/26/$31.00 ©2026 IEEE

Third, REAL performs cross-group combination to balance workload across GPUs. The cross-group scheduling process jointly considers inter-group data reuse and load balance to improve overall resource utilization. Based on these designs, we formulate an optimal scheduling problem to minimize per-epoch time. Depending on the scenario, REAL solves the problem using either Integer Linear Programming (ILP) or a greedy heuristic, resulting in two variants: REAL-L and REAL-G.

To validate the system's efficacy, we evaluate REAL on physical clusters equipped with NVIDIA H200 GPUs, scaling from single-node (8 GPUs) to distributed (32 GPUs) setups. REAL-L and REAL-G achieve 1.11×–3.26× (avg. 1.83×) and 1.10×–3.16× (avg. 1.77×) speedups over the strongest baseline on one-layer models, driven by substantial reductions in data transfer volume and redundant FLOPs; the gains narrow on two-layer models (avg. 1.39× and 1.36×), where only the first layer benefits from reuse. We further present ablation studies showing that all design components of REAL are necessary.

We make the following main contributions.

- We identify key inefficiencies in existing distributed DGNN training methods, including insufficient resource utilization, redundant transfers, and limited data reuse.
- We propose REAL, a system that integrates snapshot- and group-level reuse, and a cross-group scheduling algorithm to jointly optimize resource utilization and data reuse without incurring extra transfer overhead.
- We introduce two scheduling variants: REAL-L and REAL-G, based on ILP and a greedy heuristic, respectively. The system first attempts REAL-L and falls back to REAL-G when ILP solving exceeds the time limit.
- We evaluate REAL-L and REAL-G on six dynamic graph datasets and four DGNN models. REAL-L and REAL-G achieve up to 3.26× and 3.16× higher throughput over the strongest baseline, respectively.

II. BACKGROUND AND MOTIVATION

A. Dynamic Graph Neural Networks

Dynamic Graph Neural Networks (DGNNs) capture the evolution of graph data by stacking multiple spatio-temporal blocks. Regardless of specific architectural variations, a typical DGNN layer processes a snapshot $\mathcal{G}_t$ through two logical phases: *structural encoding* and *temporal evolution*. First, a structural encoder aggregates neighborhood information to capture spatial dependencies. Formally, for a node v, the feature aggregation is defined as:

$$\mathcal{H}_t(v) = \text{SpatioAggr}_v(\mathcal{W}_{\text{gnn}}, \{\mathcal{X}_t(u) \mid u \in \mathcal{N}(v)\}, \mathcal{X}_t(v)) \tag{1}$$

where $\mathcal{N}(v)$ denotes the neighborhood of v. Second, the aggregated features are fed into a temporal encoder to update the node's hidden state based on historical contexts:

$$h_t = \text{TemporalAggr}(\mathcal{W}_{\text{rnn}}, \mathcal{H}_t, h_{t-1}), \quad h_0 = 0 \tag{2}$$

This establishes a sequential dependency where h_t strictly relies on the previous state h_{t-1}.

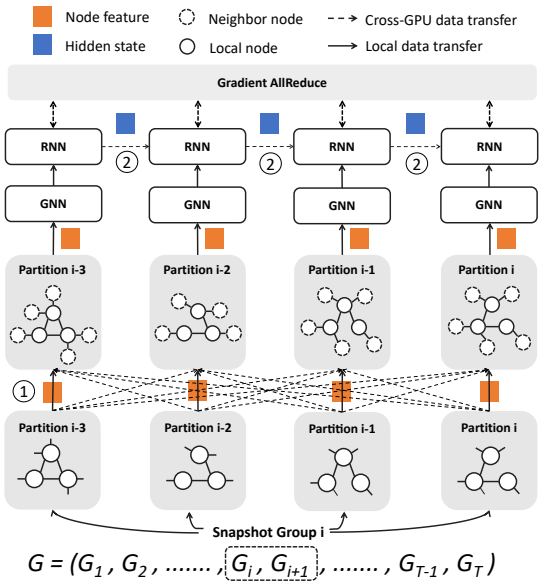


Fig. 1: **Workflow of distributed training for DGNNs.**

Representative models follow this paradigm primarily by integrating spatial aggregation with temporal recurrence. Canonical examples like WD-GCN [6], TGCN [7], and GAT-LSTM [3] combine spatial encoders (*e.g.,* GCN or GAT) with recurrent units (*e.g.,* LSTM or GRU) to jointly capture structural and temporal dependencies in node embeddings. Even parameter-centric variants like EvolveGCN [5], which evolve model weights rather than node embeddings, strictly adhere to this fundamental snapshot-based execution pattern.

B. Workflow of Distributed DGNN Training

Training DGNNs typically involves partitioning the dynamic graph sequence $\mathcal{G} = \{\mathcal{G}_1, \ldots, \mathcal{G}_T\}$ into *snapshot groups*, which serve as the basic training units. In a distributed setting, mapping the spatio-temporal dependencies defined in §II-A to multiple GPUs introduces distinct execution phases and communication patterns, as illustrated in Figure 1.

Spatial Aggregation & Neighbor Feature Transfer. To execute the spatial aggregation (Eq. 1), a GPU must access the features of all neighbors $u \in \mathcal{N}(v)$. If the graph is partitioned, the neighborhood of boundary nodes often spans multiple devices. This necessitates *Neighbor Feature Transfer* (① in Figure 1), where feature tensors must be fetched from remote GPUs before computation can proceed. This irregular, topology-dependent communication constitutes a major performance bottleneck for dense graphs.

Temporal Evolution & Hidden State Transfer. Following spatial aggregation, the temporal update (Eq. 2) requires the previous hidden state h_{t-1}. When consecutive snapshots of a node (or its relevant boundary dependencies) are assigned to different devices, the system must perform *Hidden State Transfer* (②) to synchronize states. Unlike neighbor aggregation, this introduces a strict sequential synchronization barrier, often forcing GPUs to idle while awaiting remote states.

Category	Dimension	AliGraph [37]	DistDGL [38]	ESDG [31]	DGC [33]	BLAD [32]	DynaHB [34]	REAL
Data Transfer	*No Neighbor Feature Transfer*	×	×	✓	×	✓	✓	✓
	No Hidden State Transfer	✓	✓	×	×	✓	✓	✓
	No Extra CPU-GPU Transfer	✓	✓	✓	✓	✓	×	✓
Resource Util.	*Resource-Level Parallelism*	×	×	×	×	✓	×	✓
	Load Balance	✓	✓	×	✓	×	✓	✓
Data Reuse	*Topology-Aware Reuse*	×	×	×	×	×	×	✓

TABLE I: **Comparison of existing solutions in three categories:** *data transfer (neighbor feature, hidden state, CPU–GPU transfer), resource utilization (resource-level parallelism, load balance), and data reuse (topology-aware reuse).*

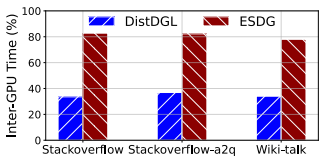


Fig. 2: *Inter-GPU data transfer time during forward pass for DistDGL and ESDG.*

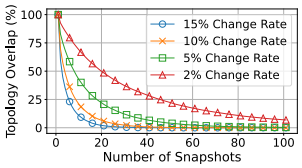


Fig. 3: *Effect of change rate and snapshot count on graph topology overlap.*

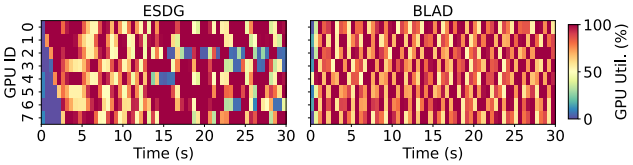


Fig. 4: *GPU utilization of ESDG and BLAD on the Stackoverflow dataset.*

Parameter Synchronization. Finally, a global *Gradient AllReduce* synchronizes model parameters across all participating GPUs after the backward pass, ensuring consistency for the next iteration.

C. Trilemma in Distributed Training of DGNNs

Efficient distributed DGNN training requires jointly optimizing three objectives: high resource utilization, minimal data transfer overhead, and maximal data reuse. Existing approaches exhibit a range of strengths and weaknesses, as shown in Table I; yet none of them address all key objectives. We next analyze these limitations according to the partitioning strategies employed.

Vertex-Based Partitioning. Vertex-based partitioning (*e.g.,* DistDGL) adopts vertex-level partitioning to achieve fine-grained load balancing across GPUs by evenly distributing nodes and their associated computations. However, this strategy fragments the graph structure within each snapshot, resulting in frequent neighbor feature exchanges across GPUs. Consequently, it incurs substantial inter-GPU data transfer during message passing. For example, the profiling of the forward pass of WD-GCN [6] on three real-world datasets shows that DistDGL incurs notable data transfer overhead, accounting for an average of 34% of the total forward pass time (Figure 2).

Snapshot-Based Partitioning. To reduce neighbor feature data transfer, ESDG performs snapshot-level scheduling by assigning each snapshot to a single GPU, ensuring that neighbor aggregation is local. However, it introduces hidden state exchange across GPUs when modeling temporal dependencies, and variations in snapshot sizes cause inter-GPU load imbalance. Moreover, the sequential dependency of RNNs exacerbates resource underutilization, as GPUs often idle while waiting for hidden states to be transferred. Using WD-GCN [6] on three real-world datasets, we observe that inter-GPU data transfer (including idle time) accounts for an average of 81% of forward pass time (Figure 2). In parallel, GPU utilization on Stackoverflow [39] dataset (Figure 4) also reveals severe underutilization of compute resources.

Snapshot Group-Based Partitioning. BLAD extends snapshot-level scheduling to snapshot groups to improve training efficiency. This approach reduces both neighbor feature transfer and hidden state data transfer, as snapshots within the same group are processed locally and sequentially. However, grouping multiple different snapshots together amplifies the imbalance problem: variations in graph size and structure accumulate within a group, making it harder to evenly distribute workload across GPUs. We also visualize GPU utilization under BLAD in Figure 4, which shows uneven utilization caused by imbalance.

Ineffective Utilization of Topological Similarity. Current approaches struggle to leverage topological similarity to accelerate distributed DGNN training, ranging from ignoring structural evolution to suffering from diminishing reuse potential and limited model applicability. For instance, DGC [33] reuses graph embeddings across epochs, but ignores topological similarity between graphs. In the single-GPU setting, PiPAD [27] adopts a global structured reuse scheme by identifying the global intersection across snapshots. However, its scalability is limited: as the number of snapshots grows, the globally shared intersection quickly shrinks. With fixed edge change rates (5%, 10% and 15%), the intersection ratio drops below 20% after 30 snapshots (Figure 3), diminishing its reuse potential. Similarly, reINC [36] employs incremental aggregation to prune redundant computations. Moreover, neither PiPAD nor reINC extends to attention-based GNNs such as GATs, as efficiently maintaining the global non-linear softmax normalization using only incremental information remains

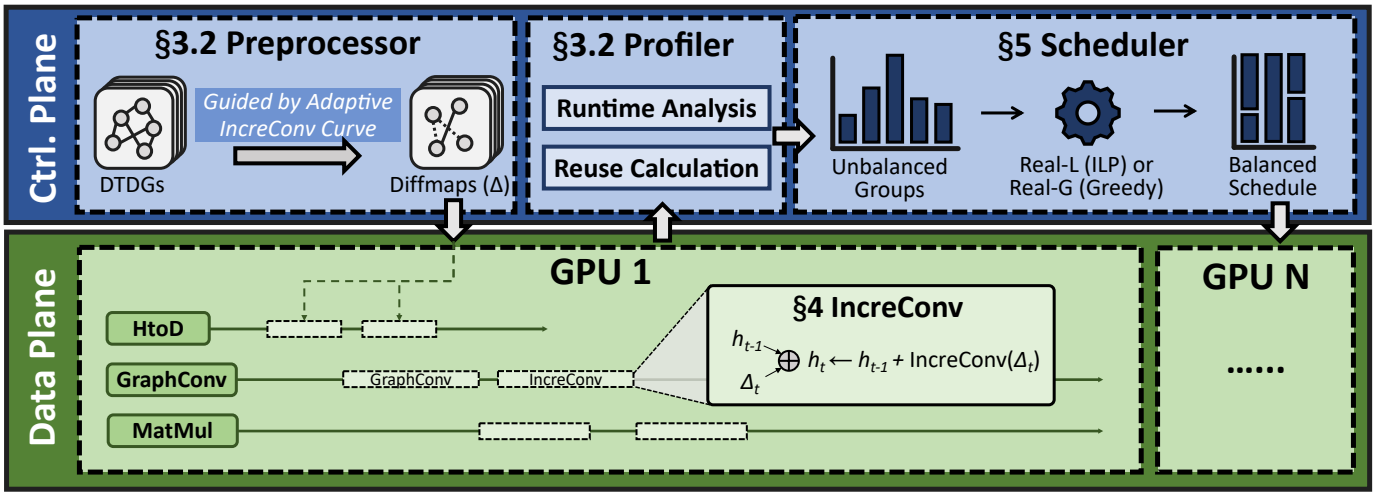


Fig. 5: **Overview of REAL.** *The system decouples the Control Plane (CPU) for preprocessing and scheduling from the Data Plane (GPU) for parallel training.*

challenging. Furthermore, in the distributed context, reINC also fails to co-optimize reuse with load balancing. It ignores the computational heterogeneity induced by varying reuse rates across snapshots, which exacerbates load imbalance and straggler effects among GPUs.

III. SYSTEM OVERVIEW

A. Design Principles

Minimize Data Transfer. REAL employs a *snapshot group-based partitioning strategy* that limits cross-GPU data transfer to only gradient synchronization. In addition, REAL proactively identifies *reusable graph structures* across snapshots to reduce redundant CPU-GPU data transfer.

Maximize Data Reuse. REAL proactively considers *both snapshot-level and group-level topology similarity*, optimizing data reuse and reducing redundant computations.

Maximize Resource Utilization. REAL improves resource utilization through two techniques: (1) a *triple-stream pipeline* that overlaps heterogeneous operations, and (2) *cross-group scheduling* that balances workloads across GPUs.

B. REAL Architecture

Figure 5 illustrates the architecture of REAL. To minimize resource contention, REAL adopts a two-plane design that decouples coordination from execution. The control plane (CPU) orchestrates graph preprocessing, profiling, and scheduling, while the data plane (GPU) focuses exclusively on distributed DGNN training. The core components are described below.

Preprocessor. The preprocessor analyzes the DTDG topology to guide efficient scheduling and execution. As an offline profiling step, the preprocessor fits degree–runtime cost curves on synthetic graphs only once during system initialization; the curves are reused across different datasets as the basis for operator selection. Then, for each snapshot $\mathcal{G}_i$ ($i > 1$), it computes a sparse difference map (`dmap`) by diffing against its predecessor $\mathcal{G}_{i-1}$, which captures localized structural changes. Finally, depending on the estimated cost from `dmap` and the fitted curves, the preprocessor selects for each snapshot an appropriate form, either the full graph or its `dmap`. For example, a four-size group can be constructed as $\mathcal{SG}_i = [\mathcal{G}_i, \texttt{dmap}_{i+1}, \mathcal{G}_{i+2}, \texttt{dmap}_{i+3}]$. Details are provided in §IV-A.

Profiler. The profiler quantifies training costs and reuse opportunities to drive the scheduler's decision-making. It first collects fine-grained training time details, including graph-level metrics such as `HtoD`, `Conv`, and `MatMul` times, as well as group-level `RNN` and backward times. To ensure reliable statistics, the profiler analyzes multiple training epochs and eliminates outliers caused by runtime noise. Based on these profiled values, it further estimates the potential reuse time $\mathcal{R}_{i,j}$ when two groups $\mathcal{SG}_i$ and $\mathcal{SG}_j$ are scheduled to the same GPU in the same iteration. This reuse time captures the overlapping data transfer and computation in `HtoD` and GNN phases. Both the profiled timeline and $\mathcal{R}_{i,j}$ table are later used by the scheduler to simulate the total epoch time.

Scheduler. The scheduler estimates the training time of an epoch by leveraging profiling statistics, incorporating both the standalone execution cost of each group and the reuse $\mathcal{R}_{i,j}$ when groups with overlapping snapshots are co-located on the same GPU. Based on this performance model, REAL supports two scheduling backends: an ILP-based solver (REAL-L) and a greedy heuristic (REAL-G), described in detail in §V. To balance scheduling quality and runtime efficiency, REAL adopts a dynamic selection strategy. At the beginning of training, REAL first attempts to solve the scheduling problem using REAL-L, and a timer is started upon invocation. If the solver fails to return a feasible schedule within a predefined threshold, the scheduler falls back to REAL-G.

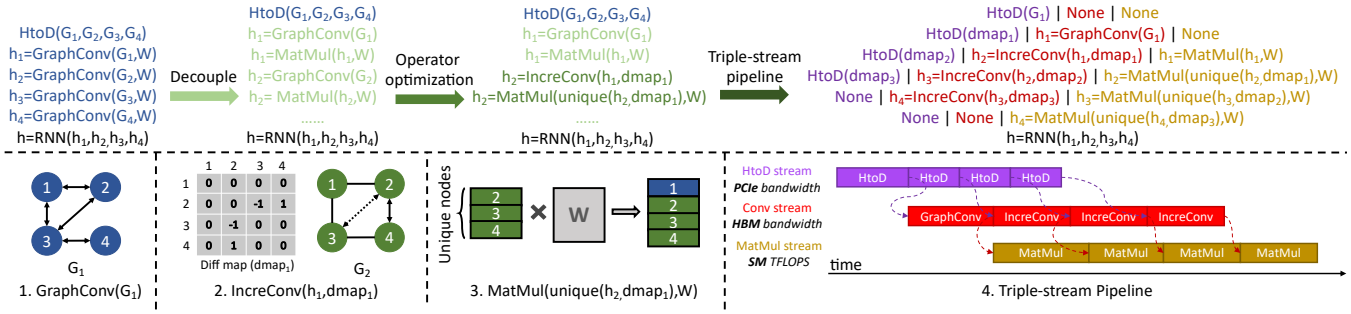


Fig. 6: **Snapshot-level data reuse.** *Using GCN with aggregation preceding transformation as an example.*

IV. TOPOLOGY-AWARE DATA REUSE

A. Type I: Snapshot-Level Data Reuse

In DTDGs, consecutive snapshots are usually highly similar: most nodes and edges remain unchanged, with only a small fraction updated. However, existing distributed DGNN systems treat each snapshot independently and rerun full `GraphConv`, causing substantial redundancy. This redundancy manifests in two main ways. First, in the aggregation phase of `GraphConv`, if a node's neighborhood remains unchanged across consecutive snapshots within the same group, its aggregated feature also stays identical, making repeated neighborhood aggregation redundant. Second, in the linear transformation phase, these identical aggregated features are repeatedly multiplied by the same weight matrix, consuming additional memory bandwidth and compute cycles. To address this, we propose a snapshot-level reuse strategy (Figure 6) that decouples aggregation and transformation in `GraphConv` for fine-grained reuse in both phases. We apply this mechanism at the first GNN layer, where identical snapshot features across time enable maximal reuse.

1) Aggregation Phase Reuse: In the aggregation phase of `GraphConv`, each node collects its neighbors' features to form a structural summary of its local neighborhood. To avoid redundant aggregation, we use a difference map (`dmap`), where each entry `dmap(i, j)` marks structural changes in the local neighborhood. We define `dmap(i, j)` = 1 if the edge (i, j) is newly added, `dmap(i, j)` = -1 if it is removed, and `dmap(i, j)` = 0 otherwise. We then define an incremental aggregation operator, `IncreConv`, to capture the differential contribution of these local changes. During `IncreConv`, each nonzero entry in the `dmap` contributes by multiplying the neighbor feature with the corresponding edge weight. Finally, the `IncreConv` output is incorporated with the prior snapshot's aggregation to obtain the updated result. Note that `IncreConv` reuses cached aggregation results from the previous snapshot, which are already retained for subsequent RNN computation, thus incurring no extra memory cost. Formally, let $h_j^{(t)}$ denote the aggregated feature of node j at snapshot t. We define the update rule as:

$$h_j^{(t)} = h_j^{(t-1)} + \underbrace{\sum_{i \in \Delta\mathcal{E}_j} \texttt{dmap(i, j)} \cdot w_{ij} \cdot h_i}_{\texttt{IncreConv}_j}, \tag{3}$$

where $h_j^{(t-1)}$ represents the cached aggregation result from the previous snapshot. $\Delta\mathcal{E}_j$ denotes the set of incident edges whose status changes at snapshot t. w_{ij} is the normalized edge weight, and h_i denotes the input feature of neighbor node i.

Degree-Aware `IncreConv` for GCN. Standard GCN normalization defines edge weights as $w_{ij} = 1/\sqrt{d_i d_j}$ based on degrees from the full snapshot. However, the differential `dmap` lacks complete neighborhood information, preventing direct weight computation. To address this, we augment each node in the `dmap` with its degrees from both the current $d^{(t)}$ and preceding $d^{(t-1)}$ snapshots. This enables us to formulate the incremental aggregation weight w_{ij} for topological updates:

$$w_{ij} = \begin{cases} +\frac{1}{\sqrt{d_i^{(t)} d_j^{(t)}}} & \texttt{dmap(i, j) = +1} \\ -\frac{1}{\sqrt{d_i^{(t-1)} d_j^{(t-1)}}} & \texttt{dmap(i, j) = -1} \end{cases} \tag{4}$$

Substituting this signed weight w_{ij} into Eq. (3) yields the final incremental embedding update. Crucially, this design significantly reduces memory overhead by eliminating the need to maintain a global edge weight table.

Softmax-Aware `IncreConv` for GAT. In contrast to GCN, where each normalized edge weight depends only on the endpoints' degrees and can thus be estimated locally from the `dmap` with lightweight degree tracking, GAT poses an additional challenge: due to the softmax operation, each attention coefficient α_{ij} depends on the *entire* neighborhood of node j. Thus, even a single edge update alters the denominator shared by all neighbors, making it impossible to derive updated attention weights solely from the `dmap`. To incrementally update GAT aggregation, we maintain for each node j the softmax denominator $\mathcal{D}_j^{(t-1)}$ and the aggregation result $h_j^{(t-1)}$ from snapshot $t-1$:

$$\mathcal{D}_j^{(t-1)} = \sum_{i \in \mathcal{N}(j, t-1)} \exp(e_{ij}^{(t-1)}) \tag{5}$$

$$h_j^{(t-1)} = \sum_{i \in \mathcal{N}(j, t-1)} \frac{\exp(e_{ij}^{(t-1)})}{\mathcal{D}_j^{(t-1)}} \mathcal{W} h_i \tag{6}$$

where $e_{ij}^{(t-1)}$ is the unnormalized attention score on edge (i, j), and $\mathcal{W}$ is the linear transformation weight. When snapshot t introduces an update set $\Delta\mathcal{E}_j$ of incident edges (with sign $+1$

for addition and -1 for deletion in `dmap`), we first compute the incremental change of the denominator:

$$\mathcal{D}_j^{(t)} = \mathcal{D}_j^{(t-1)} + \sum_{i \in \Delta\mathcal{E}_j} \texttt{dmap(i,j)} \cdot \exp\left(e_{ij}^{(t)}\right). \quad (7)$$

The previous aggregation is then rescaled and updated with the contributions of changed edges:

$$h_j^{(t)} = h_j^{(t-1)} \cdot \frac{\mathcal{D}_j^{(t-1)}}{\mathcal{D}_j^{(t)}} + \sum_{i \in \Delta\mathcal{E}_j} \texttt{dmap(i,j)} \cdot \frac{\exp(e_{ij}^{(t)})}{\mathcal{D}_j^{(t)}} \mathcal{W} h_i. \quad (8)$$

This design allows unchanged neighbors to be updated in $O(1)$ by denominator rescaling, while only the affected edges incur $O(|\Delta\mathcal{E}_j|)$ cost. Overall, the per-snapshot complexity is reduced from $O(|\mathcal{E}|)$ to $O(|\Delta\mathcal{E}|)$, where $\Delta\mathcal{E}$ is the number of changed edges.

2) Transformation Phase Reuse: The applicability of transformation reuse depends on the dependency order between the `MatMul` and `GraphConv`. *Case 1: Pre-aggregation Transformation.* In architectures where feature transformation precedes aggregation, the input node features are *time-invariant* within a snapshot group. Consequently, the projection is computed only once for the first snapshot and shared across all subsequent snapshots, eliminating redundant computations entirely. *Case 2: Post-aggregation Transformation.* When transformation follows aggregation, the input to `MatMul` depends on the temporally evolving topology. Here, we employ *selective execution*: only nodes with topologically updated neighborhoods (identified via `dmap`) trigger new `MatMul` operations, while unaltered nodes directly reuse cached results from the preceding snapshot.

3) Triple-Stream Pipeline: To further improve end-to-end efficiency, we extend our execution to three parallel CUDA streams: (i) an `HtoD` stream for host-to-device data transfer, (ii) a `Conv` stream for `Conv` operations, and (iii) a `MatMul` stream for feature transformation. These three stages stress different hardware resources: PCIe bandwidth, HBM bandwidth, and GPU compute, respectively, and thus can be effectively overlapped. The streams are coordinated via `cudaEvent` to preserve inter-operator dependencies while maximizing concurrency. Compared to conventional sequential execution, our triple-stream pipeline enables fine-grained inter-snapshot interleaving. As illustrated in Figure 6, `HtoD`, `Conv`, and `MatMul` operations from different snapshots can overlap, effectively hiding latency and improving pipeline throughput. When transformation precedes `GraphConv` (*e.g.,* GAT), the dependency becomes `HtoD` $\rightarrow$ `MatMul` $\rightarrow$ `Conv`. Here, `MatMul` is computed once and reused across snapshots, so the pipeline effectively overlaps `HtoD` and `Conv`.

B. Type II: Group-Level Data Reuse

Group-level data reuse leverages the property that model parameters remain frozen within a single training iteration. Consequently, when consecutive snapshot groups share overlapping graphs, their associated data can persist in resident GPU memory across groups. This mechanism effectively eliminates redundant `HtoD` transfers and expensive `GraphConv` computations for the overlapping portions. To fully exploit this opportunity, the REAL scheduler adopts a similarity-driven scheduling strategy. Instead of adhering to a naive order that results in disjoint pairs (*e.g.,* processing $[\mathcal{G}_1, \mathcal{G}_2]$ followed by $[\mathcal{G}_3, \mathcal{G}_4]$), the scheduler proactively reorders the execution sequence to prioritize snapshot overlap across consecutive groups (*e.g.,* chaining $[\mathcal{G}_1, \mathcal{G}_2]$ with $[\mathcal{G}_2, \mathcal{G}_3]$). This similarity-aware rescheduling constructs a reuse-centric execution chain that significantly reduces I/O and compute overhead while maintaining workload balance. The concrete *cross-group scheduling* algorithm is detailed in § V.

V. CROSS-GROUP SCHEDULING

A. Optimization Objectives

We consider a set of snapshot groups with standalone execution times $\mathcal{SG} = \{t_1, \ldots, t_n\}$, a set of available GPUs (devices) $\mathcal{D} = \{1, \ldots, D\}$, and a sequence of training iterations $\mathcal{I} = \{1, \ldots, m\}$. The primary objective of REAL is to minimize the total per-epoch training time $T = \sum_{i \in \mathcal{I}} T_i$, where T_i denotes the duration (makespan) of iteration i. We define binary decision variables $x_{k,i,j} \in \{0, 1\}$ to indicate whether snapshot group $k \in \mathcal{SG}$ is assigned to GPU $j \in \mathcal{D}$ in iteration $i \in \mathcal{I}$. Let t_k be the standalone execution time of group k, and $\mathcal{R}_{k_1,k_2}$ be the reuse benefit (time saving) if groups k_1 and k_2 are co-located. Considering device memory limits and solver efficiency, we set a capacity cap of two groups per GPU per iteration. Consequently, the actual processing time (load) of GPU j in iteration i, denoted as $\mathcal{L}_{i,j}$, accounts for both computation cost and reuse reduction:

$$\mathcal{L}_{i,j} = \underbrace{\sum_{t_k \in \mathcal{SG}} t_k \cdot x_{k,i,j}}_{\text{Base Comp}} - \underbrace{\sum_{k_1 < k_2} \mathcal{R}_{k_1,k_2} \cdot \mathbb{I}(x_{k_1,i,j} = 1 \wedge x_{k_2,i,j} = 1)}_{\text{Reuse Benefit}}, \quad (9)$$

where $\mathbb{I}(\cdot)$ is the indicator function. The duration of iteration i is determined by the bottleneck GPU plus synchronization overhead α:

$$T_i = \max_{j \in \mathcal{D}} \{\mathcal{L}_{i,j}\} + \alpha. \quad (10)$$

Minimizing T requires jointly optimizing the assignment x to balance loads and maximize reuse under capacity $L = 2$. This problem generalizes the classical makespan minimization on parallel machines [40] and is NP-hard.

B. REAL-L: ILP-based Scheduling

To solve the formulation efficiently, we propose REAL-L, which models the problem as an Integer Linear Programming (ILP) task. Standard formulations of Eq. (10) often involve the $\max$ operator and logical AND conditions, which are nonlinear. We linearize these components as follows:

Assignment and Capacity Constraints. We strictly enforce that every snapshot group is scheduled exactly once, and no GPU exceeds its capacity limit L:

$$\sum_{i\in\mathcal{I}}\sum_{j\in\mathcal{D}} x_{k,i,j} = 1, \quad \forall k \in \mathcal{SG}, \tag{11}$$

$$\sum_{k\in\mathcal{SG}} x_{k,i,j} \le L, \quad \forall i \in \mathcal{I}, \forall j \in \mathcal{D}. \tag{12}$$

Reuse Linearization (McCormick Envelopes). To handle the conditional reuse term, we introduce auxiliary binary variables $y_{k_1,k_2,i,j}$ for all pairs $k_1 < k_2$. We enforce $y_{k_1,k_2,i,j} = 1 \iff x_{k_1,i,j} = 1 \wedge x_{k_2,i,j} = 1$ using standard McCormick envelope constraints:

$$\begin{aligned} &y_{k_1,k_2,i,j} \le x_{k_1,i,j}, \\ &y_{k_1,k_2,i,j} \le x_{k_2,i,j}, \qquad \forall i \in \mathcal{I}, \forall j \in \mathcal{D}, \forall k_1 < k_2. \\ &y_{k_1,k_2,i,j} \ge x_{k_1,i,j} + x_{k_2,i,j} - 1. \end{aligned} \tag{13}$$

Min-Max Formulation via Epigraph Transformation. A direct linearization of the max operator in Eq. (10) typically requires Big-M constraints, which loosen the linear relaxation and hinder solver performance. Instead, we treat T_i as a continuous decision variable and apply an epigraph-style formulation. We constrain T_i to be lower-bounded by the load of every GPU in iteration i:

$$T_i \ge \underbrace{\sum_{k\in\mathcal{SG}} t_k \cdot x_{k,i,j}}_{\text{Total Cost}} - \underbrace{\sum_{k_1<k_2} \mathcal{R}_{k_1,k_2} \cdot y_{k_1,k_2,i,j}}_{\text{Reuse Benefit}} + \alpha, \quad \forall j \in \mathcal{D}. \tag{14}$$

Since the objective is to minimize $\sum T_i$, the solver naturally tightens T_i to the maximum load among GPUs ($\max_{j\in\mathcal{D}} L_{i,j} + \alpha$) without requiring auxiliary binary indicators. This formulation significantly tightens the relaxation bound, pruning the branch-and-bound search space efficiently. We implement REAL-L using the Gurobi solver [41]. Given the NP-hardness, we set a termination criterion based on a preset optimality gap (*e.g.,* 10%) to balance schedule quality and solving time.

C. REAL-G: *Greedy-based Scheduling*

For large-scale scenarios, we employ REAL-G (Algorithm 1) as a scalable fallback upon ILP timeout. This heuristic iteratively generates candidate schedules and selects compatible groups to co-optimize reuse and load balance, thereby minimizing execution bubbles (or "wasted time").

Input. The algorithm takes as input the GPU count D, the reuse matrix $\mathcal{R}$ capturing type II reuse savings, and the group list $\mathcal{SG} = [t_1, t_2, \ldots, t_n]$ with t_i being the execution time of group i.

Group preprocessing. Line 1 initializes preprocessing by calling `Preprocess` (lines 14–16), which inserts a *zero group* ($t_0 = 0$) as a dummy option. This ensures that any group can either pair with another group or with the zero group (*i.e.*, run alone). All groups are then sorted by execution time t_i in ascending order to facilitate efficient matching.

Algorithm 1: REAL-G

Input : GPU count D, Reuse matrix $\mathcal{R}$, Snapshot group times $\mathcal{SG} = [t_1, t_2, \ldots, t_n]$
Output: Schedule strategy $\mathbb{X}$

1 Initialize $\mathcal{SG} \leftarrow$ Preprocess $(\mathcal{SG})$, $n \leftarrow |\mathcal{SG}|$, $\mathbb{X} \leftarrow \emptyset$;
2 **while** $|\mathcal{SG}| > D$ **do**
3 $targets \leftarrow$ GetCandidateTargets $(\mathcal{SG}, \mathcal{R}, n)$;
4 $W_{\min} \leftarrow \infty$;
5 **foreach** $\mathcal{T} \in targets$ **do**
6 $pairs \leftarrow$ GetPairs $(\mathcal{T}, \mathcal{R}, \mathcal{SG}, D, n)$;
7 $W \leftarrow D \cdot \max\Big(\max_j\ pairs[j].T,\ \mathcal{T}\Big)$;
8 $W \leftarrow W - \mathcal{T} - \sum_{j=1}^{D-1} pairs[j].T$;
9 **if** $W < W_{\min}$ **then** $W_{\min} \leftarrow W$;
10 Update $\mathcal{SG}, \mathbb{X}$;
11 Record rest groups in $\mathbb{X}$;
12 **return** $\mathbb{X}$;

13 **Function** Preprocess $(\mathcal{SG})$ **:**
14 Add $t_0 = 0$ to $\mathcal{SG}$ and sort ascending;
15 **return** $\mathcal{SG}$;
16 **Function** GetCandidateTargets $(\mathcal{SG}, \mathcal{R}, n)$ **:**
17 Initialize $targets \leftarrow \emptyset$;
18 **for** $i = 0$ **to** $n - 1$ **do**
19 Add $\{\mathcal{SG}[i] + \mathcal{SG}[n] - \mathcal{R}[i][n]\}$ to $targets$;
20 **return** $targets$;
21 **Function** GetPairs $(\mathcal{T}, \mathcal{R}, \mathcal{SG}, D, n)$ **:**
22 Initialize $pairs \leftarrow \emptyset$;
23 **while** $|pairs| < D - 1$ **do**
24 Initialize $head$, $tail$, $best_pair$;
25 **while** $head < tail$ **do**
26 $T \leftarrow \mathcal{SG}[head] + \mathcal{SG}[tail] - \mathcal{R}[head][tail]$;
27 **if** $|T - \mathcal{T}| < |best_pair.T - \mathcal{T}|$ **then**
28 $best_pair \leftarrow [T, head, tail]$;
29 $T < \mathcal{T}\ ?\ head{+}{+}\ :\ tail{-}{-}$
30 Add $best_pair$ to $pairs$;
31 **return** $pairs$;

Candidate target generation. Line 3 generates a set of potential makespan targets, denoted as *targets*, using `GetCandidateTargets` (lines 16–20). This procedure pairs the group with the longest remaining execution time (denoted as $\mathcal{SG}[n]$ after sorting) with other groups to project potential bottleneck durations for the current iteration.

Group selection and waste time calculation. Lines 5–9 evaluate each candidate target $\mathcal{T} \in$ *targets*. For each target $\mathcal{T}$, `GetPairs` (lines 21–31) performs a two-point search to efficiently identify the group combination that best matches $\mathcal{T}$. The algorithm then calculates the wasted time W_i (lines

Dataset	$\overline{\lvert\mathcal{V}\rvert}$	$\overline{\lvert\mathcal{E}\rvert}$	$\overline{\lvert\mathcal{V}_d\rvert}$	$\overline{\lvert\mathcal{E}_d\rvert}$	d_v	β	γ
Arxiv [44]	169.3k	1.8M	88.7k	242.6k	128	10.6	100
Products [44]	2.4M	36.7M	1.5M	5.9M	100	15.0	100
Reddit [45]	232.9k	31.9M	211.6k	5.5M	602	137.3	100
Stackoverflow [39]	2.6M	23.1M	790.3k	3.1M	128	8.9	100
Stackoverflow-a2q [39]	2.5M	10.8M	554.3k	1.5M	128	4.4	100
Wiki-talk [46]	1.1M	6.3M	182.8k	439.4k	128	5.5	100
Arxiv†	169.3k	1.8M	89.3k	245.9k	128	10.6	300
Products†	2.2M	18.6M	1.6M	3.6M	100	7.6	300

TABLE II: **Attributes of the six datasets.** *$\overline{\lvert\mathcal{V}\rvert}$ and $\overline{\lvert\mathcal{E}\rvert}$ denote the average nodes and edges per snapshot; $\overline{\lvert\mathcal{V}_d\rvert}$ and $\overline{\lvert\mathcal{E}_d\rvert}$ denote those per* `dmap`*; d_v is the node feature dimension; β and γ denote the average degree and snapshot count; † marks Arxiv/Products variants with $\gamma = 300$.*

8–9) to quantify the quality of the schedule:

$$W_i = \underbrace{D \cdot (T_i - \alpha)}_{\text{Allocated Capacity}} - \underbrace{\sum_{j \in \mathcal{D}} \mathcal{L}_{i,j}}_{\text{Total Effective Load}} \quad . \tag{15}$$

The algorithm selects the schedule that yields the minimum waste W_i, ensuring efficient group scheduling for the iteration.

Complexity analysis. The overall time complexity of Algorithm 1 is dominated by the main loop while running at $O(n^3)$, making it computationally feasible for large-scale scenarios. In practice, when n reaches the $O(10^3)$ scale, the solving time still remains within a few seconds.

D. Scheduling Strategy Selection

To balance schedule quality and solving overhead, REAL employs a dynamic timeout mechanism bounded by the estimated epoch time T_{epoch} derived from profiling. Specifically, we set the solver time limit to $\tau = 2 \times T_{epoch}$. If the ILP solver (REAL-L) does not converge within this threshold, the system seamlessly falls back to the greedy heuristic (REAL-G). This strategy ensures that the scheduling overhead is strictly bounded while maximizing the number of epochs that benefit from optimized execution, whether via the globally optimal plan from ILP or the efficient approximation from the greedy fallback.

VI. EVALUATION

A. Methodology

Testbed. Our experiments are conducted on an H200-based GPU cluster. The cluster comprises four servers, each equipped with a 192-core Intel Xeon Platinum 8558 CPU and eight NVIDIA H200 GPUs (141 GB HBM3e per GPU) interconnected via full-bandwidth NVLink (900 GB/s). Each server also provides eight ConnectX-7 NICs, delivering up to 3.2 Tbps RoCE bandwidth for multi-node training. All experiments run inside the DGL NGC container [42], which includes DGL v2.4 [21], PyTorch v2.4.0 [43], and CUDA 12.5.

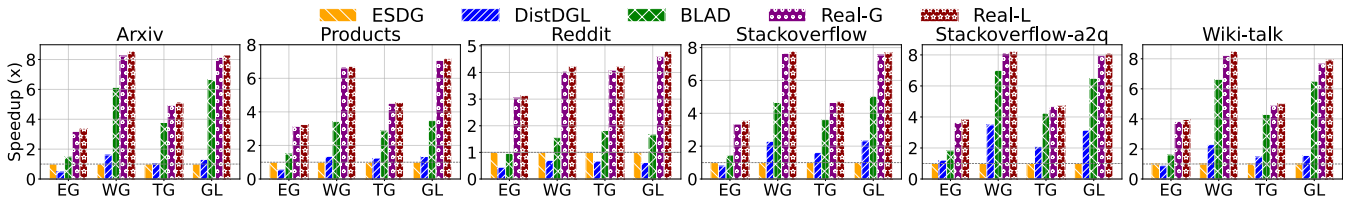
Datasets. We evaluate on six DTDG datasets (Table II), including both real-world temporal networks and synthetic dynamic graphs derived from static datasets. The real-world datasets are Stackoverflow and Stackoverflow-a2q [39], which record user interactions on the Stack Exchange website, and Wiki-talk [46], which captures edits among Wikipedia users. We also construct synthetic dynamic graphs from three static graph datasets: Arxiv [44], Products [44], and Reddit [45]. Following BLAD [32], we create snapshots by randomly changing some of the edges of the static graph. For all datasets, the snapshot group size is set to four.

Benchmark DGNN models. We use four representative DGNNs: EvolveGCN [5], WD-GCN [6], TGCN [7], and GAT-LSTM [3]. The first three models are GCN-based DGNNs, while GAT-LSTM is a GAT-based model. These models are widely used due to their effectiveness in dynamic graph learning. For each DGNN, we evaluate both one-layer and two-layer variants, where a layer consists of a spatial aggregation module (GNN) followed by a temporal update module (RNN). The hidden dimension is set to 64 across all models.

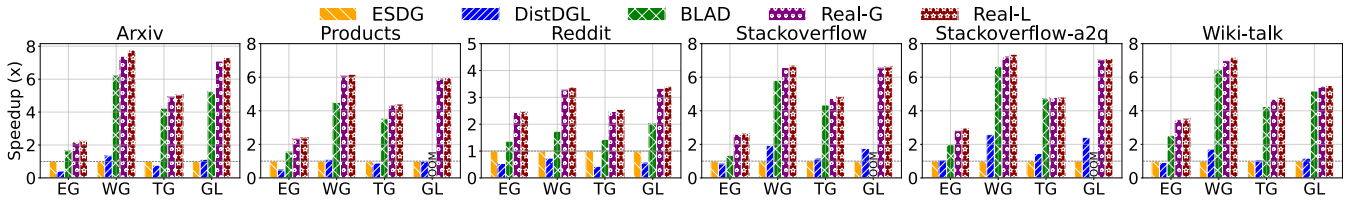
Baselines. We compare REAL-G and REAL-L with existing state-of-the-art distributed DGNN training methods, including ESDG [31], DistDGL [38] and BLAD [32]. In ESDG, snapshots in a snapshot group are evenly distributed across GPUs based on their temporal intervals. DistDGL partitions each snapshot into subgraphs and distributes them across GPUs. BLAD utilizes a two-stage pipeline to collaboratively train two consecutive snapshot groups. In contrast, REAL-G and REAL-L execute two scheduled groups sequentially. DGC [33] is excluded due to the lack of open-source code, while DynaHB [34] is an asynchronous framework and thus not comparable.

B. Experimental results

1) Overall Performance: We first compare the training speedups of all methods on a single server. We prioritize this setting to evaluate the intrinsic system performance, preventing the heavy inter-GPU communication overhead of baselines from dominating the execution time due to limited inter-node bandwidth. As shown in Figure 7, for one-layer DGNN models, REAL-L delivers substantial gains over all baselines, achieving a $3.14\times$–$8.56\times$ speedup over ESDG (avg. $5.73\times$), $2.26\times$–$7.76\times$ over DistDGL (avg. $4.68\times$), and $1.11\times$–$3.26\times$ over BLAD (avg. $1.83\times$). REAL-G also delivers consistent improvements, with a $3.05\times$–$8.24\times$ speedup over ESDG (avg. $5.57\times$), $2.23\times$–$7.42\times$ over DistDGL (avg. $4.52\times$), and $1.10\times$–$3.16\times$ over BLAD (avg. $1.77\times$). For two-layer models, REAL retains a clear advantage over all baselines, although its gains over the strongest baseline (BLAD) narrow because only the first of the two layers benefits from reuse. Specifically, REAL-L achieves speedups of $2.28\times$–$7.77\times$ over ESDG (avg. $4.88\times$), $2.79\times$–$6.57\times$ over DistDGL (avg. $4.63\times$), and $1.02\times$–$1.99\times$ over BLAD (avg. $1.39\times$). Similarly, REAL-G achieves a $2.18\times$–$7.39\times$ speedup over ESDG (avg. $4.78\times$), $2.65\times$–$6.42\times$ over DistDGL (avg. $4.53\times$), and $1.02\times$–$1.94\times$ over BLAD (avg. $1.36\times$). The gains of REAL are particularly pronounced in WD-GCN, TGCN, and GAT-LSTM, where all reuse mechanisms can be exploited. In contrast, for EvolveGCN, the weights in the `MatMul` stage change with

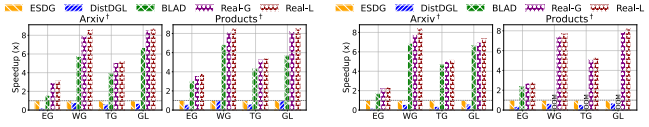


(a) One-layer DGNN models



(b) Two-layer DGNN models

Fig. 7: **Training speedup of different systems across DGNN models and datasets on a single server (8 GPUs).** *Speedup is normalized to ESDG [31]. **EG**, **WG**, **TG**, and **GL** denote EvolveGCN, WD-GCN, TGCN, and GAT-LSTM.*



(a) One-layer DGNN models (b) Two-layer DGNN models

Fig. 8: **Training speedup of different systems across DGNN models and datasets on four servers (32 GPUs).** *Speedup is also normalized to ESDG [31].*

each timestamp, preventing reuse in this phase. ESDG suffers from hidden state transfers between GPUs. The overhead is minor for EvolveGCN with small hidden states but becomes a bottleneck for larger models. DistDGL is constrained by frequent neighbor aggregation, limiting throughput scalability. BLAD improves performance mainly by reducing inter-GPU communication, but the lack of data reuse and load balancing still caps its gains. Its pipelined design also requires two data groups in GPU memory, leading to high peak usage and frequent OOM on large datasets.

We further report results on 4 servers (32 GPUs) in Figure 8. While the speedups of REAL over ESDG and BLAD remain comparable to the single-server setting, the performance gap with DistDGL widens significantly—with REAL-L achieving average speedups of 9.93× (One-layer) and 12.43× (Two-layer). DistDGL is more adversely affected in the multi-machine setting, as its vertex-level partitioning necessitates cross-machine neighbor aggregation, incurring heavy communication overhead due to limited inter-node bandwidth. In contrast, ESDG avoids cross-machine hidden state transfers when the group size matches the per-server GPU count (i.e., four), effectively confining hidden states within each server and enabling efficient inter-group data parallelism.

2) Data transfer time breakdown: Figure 9 illustrates the breakdown of per-epoch data transfer time on a single server. In ESDG, inter-GPU hidden state transfer is the dominant cost for most models except EvolveGCN, and is particularly expensive when processing large single-graph datasets such as Stackoverflow. DistDGL suffers from neighbor aggregation overhead, especially on dense graphs or with high feature dimensions, where many neighbors are on different GPUs. For example, in the Reddit dataset, with an average degree of 137 and feature dimension of 602, this results in a particularly high transfer cost. BLAD avoids both types of cross-GPU data transfer but still loads the full graph, making `HtoD` transfer a bottleneck. Its pipeline further involves hidden state transfer across processes on the same GPU, which becomes significant on datasets such as Stackoverflow and Wiki. REAL also avoids both types of cross-GPU data transfer and, within each GPU, leverages DiffMap-based reuse to greatly reduce `HtoD` volume.

3) `GraphConv` FLOPs: We profile the total `GraphConv` floating-point operations (FLOPs) within one training epoch on a single server. As shown in Figure 10, REAL significantly reduces computational redundancy. Compared to DistDGL, REAL-G achieves a FLOPs reduction of 3.88×–21.70× (avg. 11.92×), while REAL-L achieves 3.87×–20.80× (avg. 11.64×). Against ESDG and BLAD, which perform full-graph computation and thus incur identical FLOPs, REAL-G reduces operations by 1.27×–6.45× (avg. 2.95×), and REAL-L by 1.22×–5.74× (avg. 2.87×). These savings stem from skipping full `GraphConv` on structurally unchanged nodes. DistDGL's vertex-level partitioning still applies complete `GraphConv` to imported cross-GPU vertices whose outputs are never used in subsequent computation, incurring substantial redundant FLOPs. ESDG and BLAD overlook structural redundancy and perform full-graph convolution for every snapshot. In contrast, REAL leverages structural reuse to strictly limit computation to the incremental updates, effectively pruning these redundant operations.

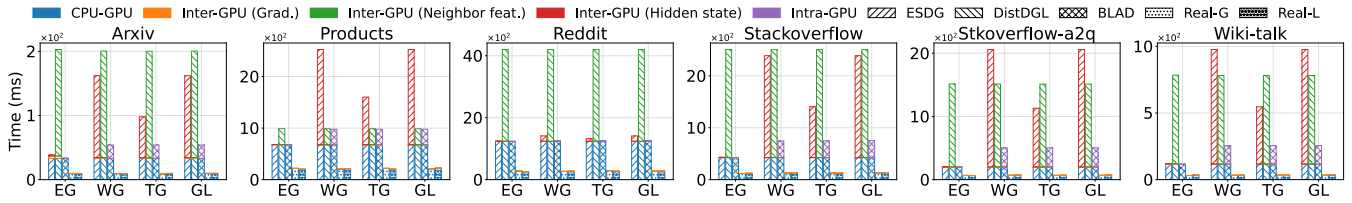


Fig. 9: **Breakdown of data transfer time.** *The time is decomposed into CPU-GPU transfer, inter-GPU communication (gradients, neighbors, hidden states), and intra-GPU transfer.*

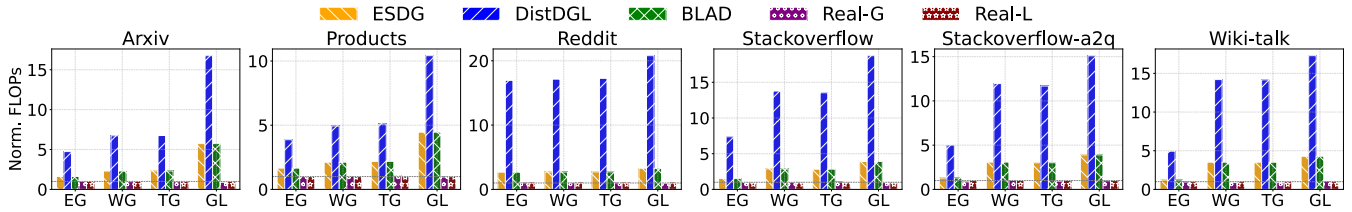


Fig. 10: **Normalized `GraphConv` FLOPs.** *FLOPs is normalized to* REAL-L.

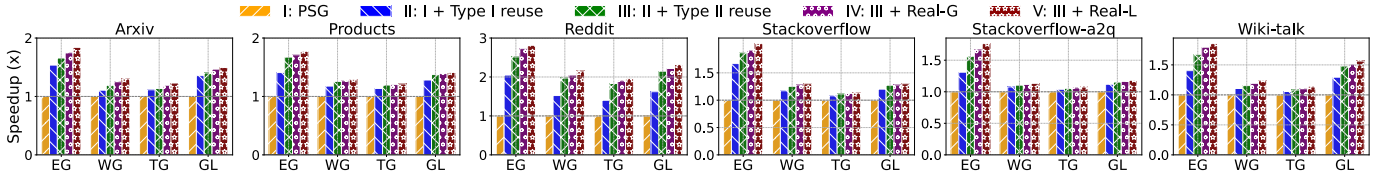


Fig. 11: **Speedup ablation across different models and datasets.** *Speedup is normalized to PSG.*

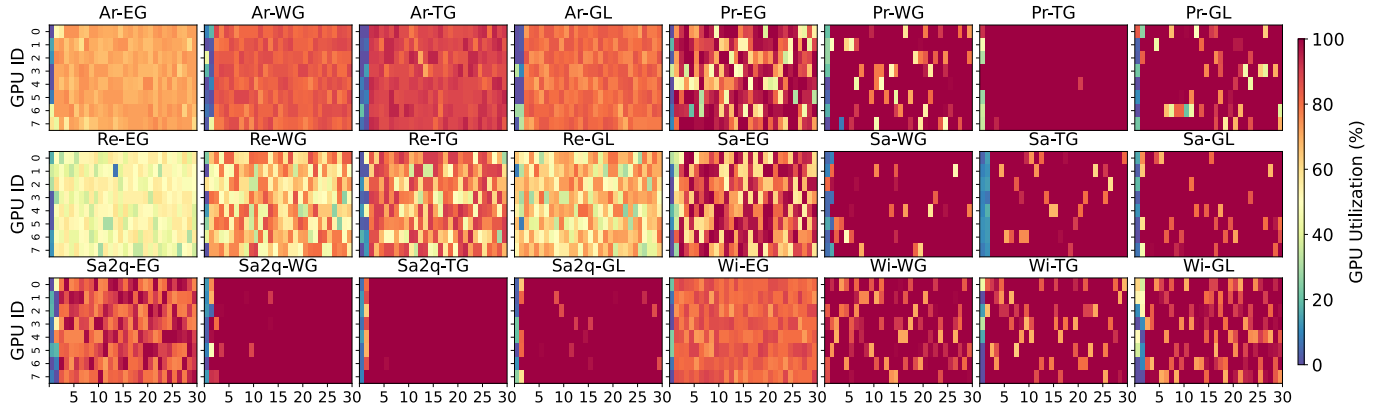


Fig. 12: **GPU utilization heatmaps of REAL-L across different DGNN models and datasets.**

4) GPU Utilization and Load Balance: We profile the utilization of all GPUs during the first 30 seconds of training. Due to space constraints, we only present the GPU utilization heatmaps of REAL-L in Figure 12. The heatmaps demonstrate that REAL-L maintains a consistently high overall GPU utilization, mainly driven by the *Triple-Stream Pipeline* in Type I reuse. By effectively overlapping `HtoD` transfers with concurrent `Conv` and `MatMul` computation streams, the system successfully masks data movement latency and eliminates execution bubbles. REAL-L also achieves a remarkably balanced workload distribution across GPUs. This is primarily due to its *cross-group* scheduling, which optimizes group assignment to reduce imbalance and alleviate bottlenecks.

C. Ablation Study

To evaluate the individual contribution of each component in REAL, we perform a stepwise ablation study normalized to the *Partition-by-Snapshot-Group* (PSG) baseline, where each GPU sequentially trains one complete snapshot group per iteration without reuse or load balancing. Figure 11 reports cumulative speedups across models and datasets.

The PSG baseline achieves 1.00× by definition, serving as the starting point. First, adding **type I reuse**—via

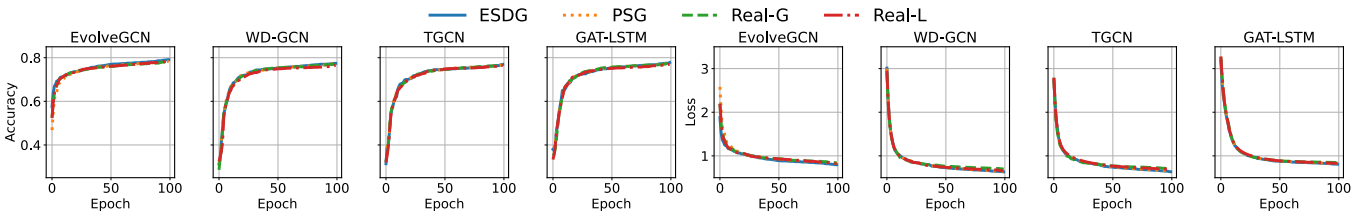


Fig. 13: **Accuracy and loss curves on the Arxiv dataset.** *The trajectories of* REAL *align closely with the baseline (ESDG) across various models, confirming statistical correctness.*

`IncreConv`, selective `MatMul`, and the triple-stream pipeline—targets intra-group redundancy. This improves performance to $1.04\times$–$2.05\times$ by effectively eliminating redundant computations and overlapping data transfers between snapshots. Next, incorporating **type II reuse**, which by default pairs two consecutive snapshot groups to maximize inter-group locality, further raises speedups to $1.05\times$–$2.52\times$. This gain stems from removing redundant `HtoD` transfers and `GraphConv` operations across group boundaries. To address the potential imbalance introduced by reuse, the greedy cross-group scheduler (**REAL-G**) is introduced; it balances workloads while preserving reuse benefits, achieving $1.06\times$–$2.73\times$. Finally, replacing the greedy scheduler with the ILP solver (**REAL-L**) delivers the highest cumulative speedups of $1.08\times$–$2.81\times$, benefiting from a global view that optimizes the entire epoch's schedule rather than making local decisions.

D. Convergence Analysis

Compared to ESDG, REAL effectively increases the batch size per iteration by a factor of $K = 2$. Given that the convergence dynamics of large-batch training are well-established [47–49], REAL achieves comparable accuracy to the baseline by proportionally scaling the learning rate. To formally validate this behavior, we analyze the convergence properties under standard assumptions (*e.g.,* L-smoothness and bounded variance). Our analysis confirms that by aggregating gradients across N GPUs and K snapshot groups, the gradient variance is reduced by a factor of $\frac{1}{NK}$, ensuring convergence to a first-order stationary point at the standard rate of $\mathcal{O}(1/\sqrt{T})$. Due to space constraints, we omit the detailed proof. We also empirically verify this consistency in Figure 13 using the Arxiv dataset, where the training trajectories (accuracy and loss) of REAL closely align with the baseline. Results on other datasets show similar trends but are omitted due to space constraints.

VII. DISCUSSION

Scope and correctness of reuse. Both snapshot-level (Type I) and group-level (Type II) reuse are applied only at the first GNN layer, where the input node features are *time-invariant* within a snapshot group; here, snapshots differ only in topology, which the degree-aware (GCN) and softmax-aware (GAT) `IncreConv` operators track incrementally while reusing the cached aggregation of the unchanged structure (§IV-A). This is exact for the softmax-aware GAT variant, which rescales the attention denominator; the GCN variant omits the second-order re-normalization of unchanged neighbors induced by degree changes, a small approximation we find empirically negligible (the accuracy and loss of REAL match the baseline, Figure 13). Beyond the first layer, the recurrent update makes a node's representation diverge across snapshots even under an unchanged neighborhood, so cross-snapshot equivalence no longer holds; all higher GNN layers and nonlinear/RNN components are therefore recomputed in full. A direct consequence is that a k-layer model accelerates only one of its k GNN layers, which explains the narrower gains on two-layer models (§VI-B). As DGNNs are typically shallow (two layers being the most common choice [50–54]), first-layer reuse still captures most of the exploitable redundancy.

Sensitivity to available reuse. REAL's speedup tracks the exploitable reuse, and the two reuse types respond differently to temporal dynamics. Type I reuse is incremental, so its savings shrink as the change rate rises or becomes bursty (Figure 3); once the difference map is no longer sparse, the preprocessor falls back to a full `GraphConv` (§III-B) rather than paying a reuse penalty. Type II reuse is insensitive to the change rate, as it reuses *entire* snapshots shared by co-scheduled groups (§IV-B); its benefit is set by the overlap the scheduler creates. REAL thus degrades gracefully toward a reuse-free but load-balanced scheduler; burstiness only concentrates the residual Type I loss in a few heavy snapshots, a skew the scheduler counteracts (§V). Larger groups expose more reuse but raise peak memory, so we fix the group size to four.

VIII. CONCLUSION

In this paper, we introduce REAL, a reuse-aware distributed training system for DGNNs. REAL directly targets the three fundamental challenges of distributed DGNN training: maximizing resource utilization, minimizing data transfer, and maximizing data reuse. REAL integrates snapshot-level and group-level reuse with cross-group scheduling, and provides two variants: REAL-L, which employs ILP to achieve globally optimized schedules, and REAL-G, a greedy fallback used when ILP solving times out. Extensive evaluation on six DTDG datasets and four DGNN models demonstrates that REAL-L and REAL-G deliver $1.11\times$–$3.26\times$ and $1.10\times$–$3.16\times$ speedup over the strongest baseline.

References

[1] L. Zhu, D. Guo, J. Yin, G. Ver Steeg, and A. Galstyan, "Scalable temporal latent space inference for link prediction in dynamic social networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 10, pp. 2765–2777, 2016. [Online]. Available: https://doi.org/10.1109/TKDE.2016.2591009

[2] L. Zhou, Y. Yang, X. Ren, F. Wu, and Y. Zhuang, "Dynamic network embedding by modeling triadic closure process," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018. [Online]. Available: https://doi.org/10.1609/aaai.v32i1.11257

[3] T. Wu, F. Chen, and Y. Wan, "Graph attention lstm network: A new model for traffic flow forecasting," in *2018 5th international conference on information science and control engineering (ICISCE)*. IEEE, 2018, pp. 241–245. [Online]. Available: https://doi.org/10.1109/ICISCE.2018.00058

[4] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, "Dyrep: Learning representations over dynamic graphs," in *International conference on learning representations*, 2019. [Online]. Available: https://openreview.net/forum?id=HyePrhR5KX

[5] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson, "Evolvegcn: Evolving graph convolutional networks for dynamic graphs," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 5363–5370. [Online]. Available: https://doi.org/10.1609/aaai.v34i04.5984

[6] F. Manessi, A. Rozza, and M. Manzo, "Dynamic graph convolutional networks," *Pattern Recognition*, vol. 97, p. 107000, 2020. [Online]. Available: https://doi.org/10.1016/j.patcog.2019.107000

[7] B. Chen, W. Guo, R. Tang, X. Xin, Y. Ding, X. He, and D. Wang, "Tgcn: Tag graph convolutional network for tag-aware recommendation," in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, 2020, pp. 155–164. [Online]. Available: https://doi.org/10.1145/3340531.3411927

[8] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," *arXiv preprint arXiv:2002.07962*, 2020. [Online]. Available: https://arxiv.org/abs/2002.07962

[9] P. Goyal, S. R. Chhetri, and A. Canedo, "dyngraph2vec: Capturing network dynamics using dynamic graph representation learning," *Knowledge-Based Systems*, vol. 187, p. 104816, 2020. [Online]. Available: https://doi.org/10.1016/j.knosys.2019.06.024

[10] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "Dysat: Deep neural representation learning on dynamic graphs via self-attention networks," in *Proceedings of the 13th international conference on web search and data mining*, 2020, pp. 519–527. [Online]. Available: https://doi.org/10.1145/3336191.3371845

[11] X. Wang, D. Lyu, M. Li, Y. Xia, Q. Yang, X. Wang, X. Wang, P. Cui, Y. Yang, B. Sun, and Z. Guo, "Apan: Asynchronous propagation attention network for real-time temporal graph embedding," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2628–2638. [Online]. Available: https://doi.org/10.1145/3448016.3457564

[12] Y. Wang, Y.-Y. Chang, Y. Liu, J. Leskovec, and P. Li, "Inductive representation learning in temporal networks via causal anonymous walks," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=KYPz4YsCPj

[13] G. Bai, C. Ling, and L. Zhao, "Temporal domain generalization with drift-aware dynamic neural networks," *arXiv preprint arXiv:2205.10664*, 2022. [Online]. Available: https://arxiv.org/abs/2205.10664

[14] J. You, T. Du, and J. Leskovec, "Roland: graph learning framework for dynamic graphs," in *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 2022, pp. 2358–2366. [Online]. Available: https://doi.org/10.1145/3534678.3539300

[15] J. Wang, W. Zhu, G. Song, and L. Wang, "Streaming graph neural networks with generative replay," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 1878–1888. [Online]. Available: https://doi.org/10.1145/3534678.3539336

[16] Y. Tian, Y. Qi, and F. Guo, "Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction," in *The Twelfth International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=82Mc5ilInM

[17] H. Li, Z. Zhang, D. Liang, and Y. Jiang, "K-truss based temporal graph convolutional network for dynamic graphs," in *Asian Conference on Machine Learning*. PMLR, 2024, pp. 739–754. [Online]. Available: https://proceedings.mlr.press/v222/li24d.html

[18] Z. Zhang, X. Wang, Z. Zhang, Z. Qin, W. Wen, H. Xue, H. Li, and W. Zhu, "Spectral invariant learning for dynamic graphs under distribution shifts," *Advances in Neural Information Processing Systems*, vol. 36, 2024. [Online]. Available: https://doi.org/10.52202/075280-0290

[19] S. Deng, H. Rangwala, and Y. Ning, "Learning dynamic context graphs for predicting social events," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1007–1016. [Online]. Available: https://doi.org/10.1145/3292500.3330919

[20] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence,*

IJCAI-18. International Joint Conferences on Artificial Intelligence Organization, 7 2018, pp. 3634–3640. [Online]. Available: https://doi.org/10.24963/ijcai.2018/505

[21] M. Y. Wang, "Deep graph library: Towards efficient and scalable deep learning on graphs," in *ICLR workshop on representation learning on graphs and manifolds*, 2019. [Online]. Available: https://arxiv.org/abs/1909.01315

[22] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. [Online]. Available: https://arxiv.org/abs/1903.02428

[23] M. Fey, J. Sunil, A. Nitta, R. Puri, M. Shah, B. Stojanovič, R. Bendias, A. Barghi, V. Kocijan, Z. Zhang, X. He, J. E. Lenssen, and J. Leskovec, "Pyg 2.0: Scalable learning on real world graphs," 2025. [Online]. Available: https://arxiv.org/abs/2507.16991

[24] H. Li and L. Chen, "Cache-based gnn system for dynamic graphs," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 937–946. [Online]. Available: https://doi.org/10.1145/3459637.3482237

[25] M. Guan, A. P. Iyer, and T. Kim, "Dynagraph: dynamic graph neural networks at scale," in *Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, 2022, pp. 1–10. [Online]. Available: https://doi.org/10.1145/3534540.3534691

[26] X. Qin, N. Sheikh, C. Lei, B. Reinwald, and G. Domeniconi, "Seign: A simple and efficient graph neural network for large dynamic graphs," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 2850–2863. [Online]. Available: https://doi.org/10.1109/ICDE55515.2023.00218

[27] C. Wang, D. Sun, and Y. Bai, "Pipad: pipelined and parallel dynamic gnn training on gpus," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2023, pp. 405–418. [Online]. Available: https://doi.org/10.1145/3572848.3577487

[28] S. Gao, Y. Li, Y. Shen, Y. Shao, and L. Chen, "Etc: Efficient training of temporal graph neural networks over large-scale dynamic graphs," *Proceedings of the VLDB Endowment*, vol. 17, no. 5, pp. 1060–1072, 2024. [Online]. Available: https://doi.org/10.14778/3641204.3641215

[29] S. Gao, Y. Li, X. Zhang, Y. Shen, Y. Shao, and L. Chen, "Simple: Efficient temporal graph neural network training at scale with dynamic data placement," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–25, 2024. [Online]. Available: https://doi.org/10.1145/3654977

[30] J. Su, D. Zou, and C. Wu, "Pres: Toward scalable memory-based dynamic graph neural networks," *arXiv preprint arXiv:2402.04284*, 2024. [Online]. Available: https://arxiv.org/abs/2402.04284

[31] V. T. Chakaravarthy, S. S. Pandian, S. Raje, Y. Sabharwal, T. Suzumura, and S. Ubaru, "Efficient scaling of dynamic graph neural networks," ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3458817.3480858

[32] K. Fu, Q. Chen, Y. Yang, J. Shi, C. Li, and M. Guo, "Blad: Adaptive load balanced scheduling and operator overlap pipeline for accelerating the dynamic gnn training," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3581784.3607040

[33] F. Chen, P. Li, and C. Wu, "Dgc: Training dynamic graphs with spatio-temporal non-uniformity using graph partitioning by chunks," *Proc. ACM Manag. Data*, vol. 1, no. 4, dec 2023. [Online]. Available: https://doi.org/10.1145/3626724

[34] Z. Song, Y. Gu, Q. Sun, T. Li, Y. Zhang, Y. Li, C. S. Jensen, and G. Yu, "Dynahb: A communication-avoiding asynchronous distributed framework with hybrid batches for dynamic gnn training," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3388–3401, 2024. [Online]. Available: https://doi.org/10.14778/3681954.3682008

[35] X. Song, T. Zhi, Z. Fan, Z. Zhang, X. Zeng, W. Li, X. Hu, Z. Du, Q. Guo, and Y. Chen, "Cambricon-g: A polyvalent energy-efficient accelerator for dynamic graph neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 1, pp. 116–128, 2021. [Online]. Available: https://doi.org/10.1109/TCAD.2021.3052138

[36] M. Guan, S. Singhal, T. Kim, and A. P. Iyer, "Reinc: Scaling training of dynamic graph neural networks," 2025. [Online]. Available: https://arxiv.org/abs/2501.15348

[37] R. Zhu, K. Zhao, H. Yang, W. Lin, C. Zhou, B. Ai, Y. Li, and J. Zhou, "Aligraph: A comprehensive graph neural network platform," 2019. [Online]. Available: https://doi.org/10.1145/3292500.3340404

[38] D. Zheng, C. Ma, M. Wang, J. Zhou, Q. Su, X. Song, Q. Gan, Z. Zhang, and G. Karypis, "Distdgl: Distributed graph neural network training for billion-scale graphs," 2021. [Online]. Available: https://doi.org/10.1109/ia351965.2020.00011

[39] Stack-Overflow, "Stack-overflow dataset," https://snap.stanford.edu/data/sx-stackoverflow.html, 2023.

[40] M. R. Garey and D. S. Johnson, *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990.

[41] Gurobi Optimization, LLC, "Gurobi - the fastest solver," 2022, accessed: 2022-01-01. [Online]. Available: https://www.gurobi.com

[42] NVIDIA, "Deep graph library (dgl) container," https://catalog.ngc.nvidia.com/orgs/nvidia/containers/dgl, 2024, accessed: 2025-08-20.

[43] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library*. Red Hook, NY, USA: Curran Associates Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[44] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *Advances in neural information processing systems*, vol. 33, pp. 22 118–22 133, 2020. [Online]. Available: https://arxiv.org/abs/2005.00687

[45] Reddit, "Reddit dataset," https://www.reddit.com/, n.d.

[46] A. Paranjape, A. R. Benson, and J. Leskovec, "Motifs in temporal networks," in *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining*, ser. WSDM '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 601–610. [Online]. Available: https://doi.org/10.1145/3018661.3018731

[47] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: Training imagenet in 1 hour," 2018. [Online]. Available: https://arxiv.org/abs/1706.02677

[48] S. McCandlish, J. Kaplan, D. Amodei, and O. D. Team, "An empirical model of large-batch training," 2018. [Online]. Available: https://arxiv.org/abs/1812.06162

[49] S. Ghadimi and G. Lan, "Stochastic first- and zeroth-order methods for nonconvex stochastic programming," 2013. [Online]. Available: https://doi.org/10.1137/120880811

[50] C. Li, R. Liu, J. Fu, Z. Zhao, H. Duan, and Q. Zeng, "Dgnn-mn: Dynamic graph neural network via memory regenerate and neighbor propagation," *Applied Intelligence*, vol. 54, no. 19, p. 9253–9268, Jul. 2024. [Online]. Available: https://doi.org/10.1007/s10489-024-05500-3

[51] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[52] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," 2020. [Online]. Available: https://arxiv.org/abs/2006.10637

[53] H. Yuan, Q. Sun, X. Fu, C. Ji, and J. Li, "Dynamic graph information bottleneck," 2024. [Online]. Available: https://arxiv.org/abs/2402.06716

[54] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," 2018. [Online]. Available: https://arxiv.org/abs/1706.02216